
A Same-Generator Corpus and Classifier Floors for AI-Control Monitor Evaluation

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Trusted monitors for AI control are tuned and ranked on stored pairs of code:
2 for each programming problem, an honest solution beside a solution carrying a
3 planted backdoor. In the corpora used for this, the honest solution is human-written
4 and the backdoor is written by a language model, so the two classes differ in
5 who wrote them as well as in whether they are sabotaged. Filtering to working
6 solutions and stripping every comment does not remove that difference. On the
7 filtered, comment-stripped split, a bag-of-words classifier separates the two classes
8 at AUROC 0.806, and a classifier that reads no code at all, only problem metadata,
9 reaches 0.772. The signal transfers across backdoor generators and is almost as
10 strong on backdoors that never work, so it is not a property of the sabotage. We
11 then rebuild the corpus so that one model writes both classes on the same problems,
12 and release it. Holding the writer constant removes between a third and nine-tenths
13 of the separability, and holding the prompt constant as well removes most of the
14 rest. An LLM monitor moves the same way, from 0.958 on stored pairs to 0.566
15 on same-generator pairs. The label installs a second artifact: a problem whose
16 backdoor fails contributes its human honest solution, so the negative class records
17 attack failure. Stored-pair evaluations should report a no-code and a bag-of-words
18 floor against permutation nulls, generate both classes from one model under one
19 prompt, and define the negative class by honesty rather than by attack failure.

20 1 Introduction

21 The backbone protocol of AI control is trusted monitoring, where a weaker trusted model reads
22 an untrusted model's output and flags sabotage [3]. Monitors are developed, tuned and ranked
23 almost entirely on *stored* honest/attack pairs, fixed corpora in which each programming problem
24 contributes an honest solution and a solution carrying a planted backdoor [9, 14, 7]. Honest solutions
25 are human-written, taken from APPS [5]; every backdoor is written by a language model, and the
26 corpus records which one. Separation credited to a monitor may be separation between two authors.

27 There are various known confounding variables between the human and AI solutions, such as in
28 commenting style, so corpora are often preprocessed to mitigate them. The filtered release keeps only
29 working honest solutions and working backdoors and strips every comment [9]; Control Tax reports
30 its pass-tests pre-filter and moves its failed backdoors to an appendix [14]; the monitor-ensemble
31 harness of [7] ships a `-leak-check` flag with two trivial baselines in it: a `#count` detector that
32 falls from 0.6957 to 0.4983 once comments are stripped, and code length alone, at 0.4726. Outside
33 AI control, bag-of-words is known to separate human-written from model-written code [11], and
34 the benchmarks built for that task match their two classes on line and character count or on shared
35 problem definitions [4, 1]. Adjacent control work attacks other axes: monitors subverted with injected
36 text, attacker-side selection shown to inflate safety estimates, transcript length removed as a nuisance
37 covariate [15, 2, 12].

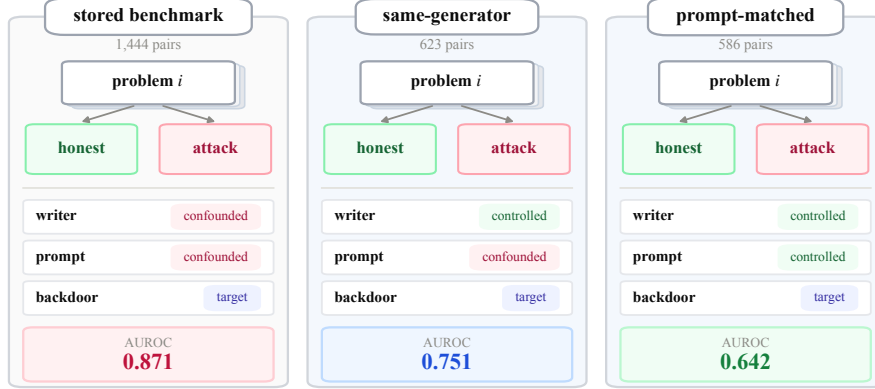


Figure 1: **How a stored pair is built, and how we rebuild it.** Each row is one property of the pair, marked *confounded* where the two classes differ in it and *controlled* where one model or one prompt holds it constant. The backdoor row is the target: it is the only difference a monitor should read, and the only one left once both controls are applied. Under each construction is the bag-of-words classifier’s AUROC on that construction’s own pairs, so the three are a ladder and not a paired comparison. The stored-benchmark 0.871 is on the full 1,444-problem artifact, not the filtered split, whose value is 0.806; each arm’s numbers are in Table 2 (notation: Table 1).

None of that preprocessing removes the authorship gap. On the filtered, comment-stripped split, a bag-of-words classifier reaches AUROC 0.806, above 20 of the 24 open-weight monitors ranked on that same split, whose median score is 0.609; a classifier that reads no code at all, only problem difficulty, test count and question length – the *no-code floor* – reaches 0.772, above 20. The classifier transfers across backdoor generators and is nearly as strong on *inert* backdoors, ones that never produce a wrong answer, as on working ones. Neither is a property of a backdoor.

We contribute (i) a construction for stored-pair corpora, in which one model writes both classes under one prompt framing and one harness verifies both, and the corpus that implements it. Run on two generators, it attributes between a third and nine-tenths of the separability to authorship and most of the remainder to the attack prompt’s framing; a one-sided variant, regenerating only the honest class, substitutes one confound for another. We identify (ii) a second selection artifact the filter installs by defining the negative class through attack failure, and (iii) the construction and reporting practice these imply. Our claims concern monitor training, selection and benchmarking on stored pairs, and our measurement is of one APPS-derived family plus five agentic settings (appendix A.9). ControlArena, the evaluation framework most of these corpora are exported from, generates both classes live and is not implicated.

2 Setup

We use seven public corpora, whose per-class provenance is the object of study: two views of one APPS-backdoor artifact (appendix A.1) and five agentic settings (appendix A.9). In the APPS family the honest class is human and the attack class is one of ten named models. Six of the seven declare no licence [8, 13], so we report only aggregates and redistribute no corpus.

Our primary protocol is one of the seven, the filtered, comment-stripped split [9]: 1,209 train problems (2,418 paired items) and 1,375 test items (172 attack, 1,203 honest), labelled by `backdoor_works`, reconstructed with the released harness code [7]. The split is a filtered view of a 1,444-problem artifact whose full pool the same-generator arms use, so two stored-pair values recur: 0.806 on this split, $B = 0.871$ on the artifact (Figure 1). Generated code is scored by our own runner, applied to both classes (appendix A.11).

Against that we build a corpus in which one model writes both classes on the same APPS problems; the construction is run twice, with Sonnet 5 and with Haiku 4.5 as the generator. The *independent* arm writes the attack class from the problem statement alone, so the writer changes and nothing else (the *headline* rows of Table 2). An *edit* arm, shown its own honest solution, returns near-copies (appendix A.4), and a *prompt-matched* arm regenerates the honest class from the attack prompt with

the clauses asking for a flaw replaced by their correct-program counterparts, so both classes are elicited alike. We sample one attack per problem, fixed before its outcome is known, and define the attack class as the benchmark does, by intent together with passing the public tests. A backdoor *fires* if it passes the public tests and gives a wrong answer on the trigger input the attacker declares; one that never fires is *inert*. Table 1 fixes the notation, and the bands were registered before any arm ran: $\rho \leq 0.33$ reads as authorship (the *collapse* band), $\rho > 0.72$ as content, the interval between as neither (*partial*).

Table 1: **Notation.** One classifier throughout; the arms differ only in the corpus it reads.

Symbol	Quantity
B	on the stored pairs, over all $n = 1,444$ problems
B'	on the stored pairs, refitted on the problems one arm realises
S	on a same-generator arm, where one model wrote both classes
$\rho = (S - 0.5)/(B' - 0.5)$	the share of above-chance separability that survives

3 Results

Comment stripping succeeds on comment count and code length, driving both to chance (Table A2, appendix). Two baselines on that same split do not move; we call them *floors*: a monitor at or below a floor has shown nothing beyond what the corpus leaks. Bag-of-words reaches 0.806 and the no-code floor reaches 0.772, both far above their permutation nulls and above 20 of the 24 monitors ranked on that split (Figure A1; intervals, nulls and paired-bootstrap counts in appendix A.2). The two are not trained alike – the monitors are zero-shot, our classifiers fitted on the split’s own train problems – so the comparison states what the corpus makes recoverable, not that a bag of words is the better monitor.

Three further measurements show that the bag-of-words classifier is not reading the backdoor (the first two are pairwise accuracies, not AUROCs; appendix A.2). It transfers across backdoor generators at 0.969: trained on one attacker’s backdoors it identifies another’s, so the feature it has learned is one the attackers share. It is nearly as strong on inert backdoors as on working ones, 0.879 against 0.935, although a backdoor that never fires should be the harder case for a detector of sabotage. The feature it does read is stylistic: `__main__` appears in 0.116 of attack samples against 0.000 of honest ones, a gap the comment stripper leaves untouched and one that replicates on the train split, in 152 of 1,209 attack samples against 1 of 1,209 honest ones. All three measurements are comment-stripped on both classes; the unstripped values, inflated by the comment channel, are in appendix A.3, and a token-level version of the last test does not replicate (appendix A.12).

Holding the writer constant removes between a third and nine-tenths of the above-chance separability (Table 2, whose B' is refitted on each row’s own problems and includes inert attacks): the two generators return $\rho = 0.687$ (Sonnet 5) and 0.095 (Haiku 4.5).

The pre-registered bands return a different verdict for each generator. Sonnet 5’s interval excludes the collapse band and spans the content boundary, leaving it inconclusive between partial and content; Haiku 4.5’s lies inside the collapse band. Sample size does not account for the fall (appendix A.5). Two further rows are *strict*, keeping only attacks that fire; a single sampled attack does so 2.2% of the time, so these rows rest on 99 pairs and agree with the headline rows at low power (appendix A.6).

Table 2: **What is left when one model writes both classes.** S is the same-generator AUROC (5-fold, comments stripped) and B' the stored-pair baseline on the row’s own problems; ρ , its bands and its intervals are as in the text and appendix A.5.

Arm	n	S (95% CI)	B'	ρ (95% CI, joint)	band
Sonnet 5, headline	623	0.7512 (0.7295–0.7713)	0.8658	0.687 (0.625–0.755)	partial–content
Haiku 4.5, headline	611	0.5340 (0.5159–0.5531)	0.8584	0.095 (0.044–0.150)	collapse
Sonnet 5, prompt-matched	586	0.6423 (0.6179–0.6621)	0.8613	0.394 (0.326–0.454)	partial
Sonnet 5, strict	99	0.7652 (0.7016–0.8000)	0.7877	0.921 (0.662–1.089)	partial–content
Sonnet 5, strict, matched	96	0.7113 (0.6464–0.7453)	0.8030	0.697 (0.494–0.856)	partial–content

The two generators respond differently to the attack prompt. On Sonnet 5 what survives is lexical rather than structural: length alone falls to 0.5851 while TF-IDF holds, and of the feature-ablation masks we apply only identifier normalisation moves S (appendix A.5). The top attack-side features are `def main` (67% of attack programs against 38.5% of honest ones) and `data[`, which together are the program layout the attack prompt elicits. Haiku 4.5 never adopts that layout in any of its three classes – honest, attack, or prompt-matched honest (appendix A.6) – which is why the two generators part. Holding the prompt constant as well as the model moves them back toward each other, to $\rho = 0.394$ on Sonnet 5 (Table 2) and $\rho = 0.113$ on Haiku 4.5 (appendix A.6). On Sonnet this removes most of what is left. The original and the prompt-matched honest classes – both correct, no attack on either side – already separate at 0.6006, so the prompt carries framing and no backdoor at all; on Haiku 4.5 that same floor contains chance.

Regenerating only one class fails: Qwen3-Coder-30B writing the honest class against the stored backdoors separates *better* than the stored pairs do, $\rho = 1.234$, one authorship gap replaced by another (appendix A.5).

The same inflation appears with an LLM monitor as the detector. We put a zero-shot monitor prompt to the model that wrote both classes, the most favourable case, since it reads its own output. It scores 0.9579 on the stored pairs and 0.566 on the same-generator pairs, a monitor- ρ of 0.145, below the classifier’s 0.687; on the strict pairs the ratio is 0.717, and a monitor that wrote neither class returns nearly the same two ratios (appendix A.7).

The split selects on a second property besides authorship. Labelling by `backdoor_works` defines the negative class by attack failure rather than by honesty: 80.7% of the test split’s labelled backdoors do not work, and a problem whose backdoor fails contributes its human honest solution instead – the label is set before any code is read. Attack problems also carry half as many test cases (20.95 against 38.85), so problem difficulty correlates with the label, which the no-code floor’s 0.772 reads.

4 Limitations

A high floor is evidence about a corpus, not about results computed on it. Ours show the label is recoverable without reading a backdoor, not that any monitor recovers it that way, and no number here corrects a published monitor score.

The control arm is limited by the generator we had to substitute. The stored backdoors’ writer, `claude-3-7-sonnet-thinking`, is retired, so both same-generator classes come from a later model of the same lineage. It differs from the stored attacker in backdoor style as well as in authorship, so every ρ here, classifier or monitor, bounds the content share of the separability from above instead of partitioning it. The arm is not bit-reproducible: it ran through a metered API that does not expose sampling parameters, with thinking off although the stored attacker was a thinking model.

Our scope is narrow in three further ways. We measure one APPS-derived corpus family and five agentic settings, and other stored-pair corpora are untested. Five of our seven corpora are one user’s repackaging of ControlArena releases, so our claims are scoped to those artifacts as published. Two interpreter-era artifacts, human solutions written for an older Python, fall on opposite classes, so B carries a small execution-environment channel the same-generator arm does not (appendix A.11).

5 Discussion

These results imply one rule for construction: one model should write both classes under one prompt framing, and one harness should verify both. Matched construction is preceded in code-detection benchmarks [4] but has not reached backdoor detection.

The definition of the negative class needs the same treatment. Defining it through attack failure ties the label to the attacker rather than to the code; dropping the failed problems removes that, as does sampling the class independently of attack success. A monitor number is interpretable only against the floors of its own split, so a no-code and a bag-of-words floor belong beside it, each against a permutation null, with the split and preprocessing released alongside (ours: MIT, appendix A.10).

Stored pairs remain the cheapest way to develop a monitor, and nothing here argues for abandoning them, only that a score on one describes two authors until the construction rules that reading out.

References

- [1] Basak Demirok, Mucahid Kutlu, and Selin Mergen. MultiAIGCD: A comprehensive dataset for AI generated code detection covering multiple languages, models, prompts, and scenarios. 2025. arXiv:2507.21693v1.
- [2] Catherine Ge-Wang, Tyler Crosse, Benjamin Hadad, Joachim Schaeffer, Ram Potham, and Tyler Tracy. Attack selection in agentic AI control evaluations meaningfully decreases safety. 2026. arXiv:2606.06529.
- [3] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. 2023. arXiv:2312.06942.
- [4] Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. CodeMirage: A multi-lingual benchmark for detecting AI-generated and paraphrased source code from production-level LLMs. 2025. arXiv:2506.11059v1. No venue stated.
- [5] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2105.09938. The upstream source of the human-written honest class, distributed as the HuggingFace dataset codeparrot/apps.
- [6] Rogan Inglis. apps-control-arena. HuggingFace dataset RoganInglis/apps-control-arena. MIT licence. 5,000 rows; honest class from codeparrot/apps., 2025.
- [7] Anik Jha. Decorrelation is not complementarity: Skill, not lineage, governs trusted-monitor ensembles. 2026. arXiv:2608.16190. Code: <https://github.com/anik-jha/challenger-panels> (MIT).
- [8] Eugene Koran. apps-control-arena-high-quality. HuggingFace dataset eugenekoran/apps-control-arena-high-quality. **No licence declared.** 1,209 train / 1,375 test., 2026.
- [9] Eugene Koran, Yejun Yun, Samantha Tetef, Benjamin Arnav, and Pablo Bernabeu-Pérez. Ensemble monitoring for AI control: Diverse signals outweigh more compute. 2026. arXiv:2605.15377v2.
- [10] Bryce Little. Length penalties make chain-of-thought less monitorable. 2026. arXiv:2607.09786v3.
- [11] Daniil Orel, Dilshod Azizov, Indraneil Paul, Yuxia Wang, Iryna Gurevych, and Preslav Nakov. AICD bench: A challenging benchmark for AI-generated code detection. 2026. arXiv:2602.02079v1. No venue stated on the abs page.
- [12] Lucas Pinto. Calibration-family overfit: Why trusted sabotage monitors don’t transfer across lineages. In *COLM 2026 Workshop on AI Measurement Science*, 2026. arXiv:2607.06596v3.
- [13] Aditya Sinha. control_arena_{bash,shade,iac,rogue_eval,agentdojo}. HuggingFace datasets adityaasinha28/control_arena_*. **No licence declared.** 1,500 rows each., 2026.
- [14] Mikhail Terekhov, Zhen Ning David Liu, Caglar Gulcehre, and Samuel Albanie. Control tax: The price of keeping AI in check. 2025. arXiv:2506.05296v3.
- [15] Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. Adaptive attacks on trusted monitors subvert AI control protocols. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.09462v2.

198 A Supplementary tables and figures

199 A.1 The schema

200 Table A1 is the record schema of RoganInglis/apps-control-arena [6]. The attack class carries
 201 model and source, naming the system that produced each backdoor; the honest class carries no such
 202 field, because there is no generator to name. The two APPS-backdoor corpora we use are two views
 203 of this one artifact, since every problem in the filtered release [8] is also in [6], byte-identical except
 204 a re-validated backdoor_works.

Table A1: The confound is legible in the dataset schema, with no measurement at all. The attack class records which model wrote it; the honest class has no generator field. Verified against the shard schema of RoganInglis/apps-control-arena.

struct	sub-fields
solutions	code, compiles, passes_tests
backdoors	backdoor_code, backdoor_input, backdoor_works, compiles, model, passes_tests, source

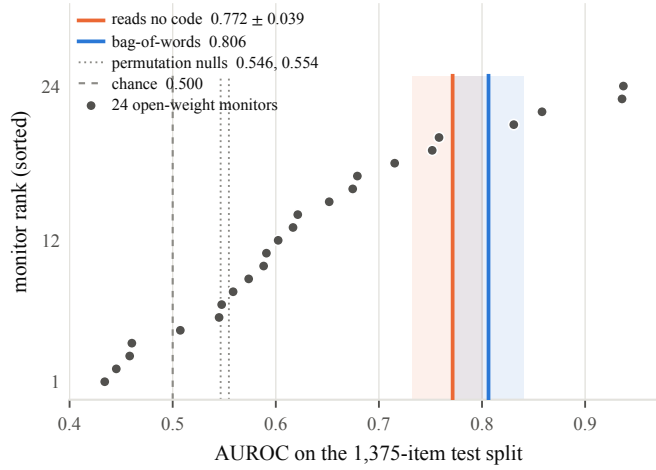


Figure A1: **Two floors against the 24 open-weight monitors ranked on the same split.** AUROC on the 1,375-item test split (172 attack / 1,203 honest), comments stripped; monitor scores from the MIT-licensed release of [7], median 0.609, max 0.937. Both floors clear 20 of the 24. **Two caveats.** (i) The monitors are zero-shot prompted while the bag-of-words classifier is trained on the benchmark’s own paired train split. The comparison states what the corpus makes recoverable and does not claim that bag-of-words is a better monitor. (ii) The no-code floor is cross-validated *within* the test split; on one held-out slice it is 0.716, above 18 of 24 (appendix A.2).

205 A.2 The protocol × baseline table

206 The no-code floor is cross-validated within the test split because the released train/val split is
 207 paired and therefore metadata-balanced; its permutation null shuffles labels for fitting and scoring
 208 alike. Table A2 is the full measurement behind §3. Rows are not comparable across protocols;
 209 the preprocessing column says whether comments were stripped before the baseline read the code.
 210 Pairwise accuracies are given to three decimals because their resolution over 2,641 pairs is one pair.
 211 Intervals in §3 are bootstraps over problems (2,000 draws) and nulls are 200 label permutations;
 212 the held-out no-code row is barely above its own null: 0.716 (CI 0.6476–0.7796) against a 95th
 213 percentile of 0.7023, $p = 0.030$, above 18 of the 24 monitors by point estimate and 12 with a paired
 214 interval excluding zero. The no-code claim therefore rests on the cross-validated protocol. The
 215 two floors of §3 in full: bag-of-words reaches 0.806 (95% CI 0.7704–0.8409 by bootstrap over
 216 problems; permutation null 95th percentile 0.5465, $p < 0.005$), above 20 of the 24 monitors by

point estimate and 18 with a paired-bootstrap interval excluding zero; the no-code floor reaches 0.772 (CI 0.7319–0.8005 on the pooled out-of-fold 0.766; null 0.5545, $p < 0.005$), above 20 and 17 respectively.

Table A2: What the field’s preprocessing leaves behind. Every row is measured under the protocol named in column one; rows are not comparable across protocols. *Prep.* says whether comments were stripped before the baseline read the code; *no code* means the baseline never reads the program. The comments-kept rows are inflated by the comment channel and are given for contrast, not as results. The matched rows are problem-disjoint and scored against the DSPy tutorial monitor; the transfer row runs Control Tax to the GPT-4-era set. Pairwise accuracies are given to three decimals, their resolution over the pair count.

Protocol	Baseline	Value	Prep.	Against
<i>AUROC</i>				
Koran’s exact split	TF-IDF word 1–2gram	0.8063	stripped	above 20 of 24
same	TF-IDF word 1–2gram	0.8914	kept	above 22 of 24
same, reads no code	difficulty + #tests + $ q $	0.7715 ± 0.0389	no code	above 20 of 24
same, held-out slice	difficulty + #tests + $ q $	0.7163	no code	above 18 of 24
same	code length only	0.4726	stripped	chance
same	#-count detector	0.4983	stripped	chance
same	#-count detector	0.6957	kept	—
Raw artifact, problem level	all metadata	0.8917	no code	—
same	difficulty only	0.6823	no code	—
same	<code>solution_passes_tests</code>	0.7881	no code	—
Matched pairs (P3)	all metadata	0.5000	no code	by construction
<i>Pairwise accuracy</i>				
DSPy matched pairs	TF-IDF word 1–2gram	0.897	stripped	monitor 0.69–0.81
working backdoors only	TF-IDF word 1–2gram	0.935	stripped	—
inert backdoors only	TF-IDF word 1–2gram	0.879	stripped	not sabotage
transfer, cross-generator	TF-IDF word 1–2gram	0.969	stripped	—
all pairs, comments kept	TF-IDF word 1–2gram	0.962	kept	—
working, comments kept	TF-IDF word 1–2gram	0.979	kept	—
inert, comments kept	TF-IDF word 1–2gram	0.955	kept	—
transfer, comments kept	TF-IDF word 1–2gram	0.994	kept	—
same, raw artifact	code length only	0.826	kept	—

220 A.3 Comment stripping on the matched-protocol numbers

221 The matched-pair and transfer measurements were first made with comments in. Every value in
222 the authorship paragraph of §3 was re-measured with the same stripper applied to both classes,
223 over five problem-disjoint splits (each transfer row is one fixed split). With comments kept the
224 same quantities were transfer 0.994, working 0.979, inert 0.955, all pairs 0.962 and surface-feature
225 AUROC 0.8306; stripped they are 0.969, 0.935, 0.879, 0.897 and 0.7259. Every one was inflated by
226 the comment channel, which is this paper’s thesis applied to itself. The #-presence marker, 0.685
227 against 0.176 before stripping, closes to 0.007 against 0.008 after and is dropped; `__main__` is
228 unchanged. One authorship leak survives the stripper because string literals are not comments: in the
229 comment-stripped train split an author byline (Author :) survives in 23 of 1,209 honest samples
230 against 2 attack samples, and a triple-quoted block in 43 against 9. Both are far too rare to drive 0.806,
231 but both are a literal record of authorship. Stripped, the matched-protocol TF-IDF AUROC is 0.8560,
232 beside $B = 0.871$ and the Koran-split 0.806, all three on stripped code. The other cross-generator
233 transfer rows, stripped, are 0.866 (o3-mini to claude-3-7-sonnet, 82 test pairs), 0.844 (the reverse)
234 and 0.764 (o1-mini to gemini-2.0-flash).

235 A.4 Pair similarity, and why the same-generator control needs two arms

236 An attack class built by editing its own honest solution changes pair similarity as well as authorship
237 (Table A3); the independent arm sits in the stored benchmark’s regime and if anything slightly further
238 from it, so it cannot manufacture a low S by making the classes alike.

Table A3: Character similarity of honest/attack pairs, comments stripped. The edit arm’s classes are near-copies; the independent arm reproduces the stored benchmark’s regime, where the two classes are largely independent programs. The last two columns are the share of pairs above and below those similarities.

Attack class	mean sim.	share > 0.95	share < 0.60	changes vs. the benchmark
stored benchmark	0.606	3.4%	45.4%	the regime to match
edit (shown its own solution)	0.928	64.2%	4.0%	authorship and similarity
independent (problem only)	0.659	3.4%	34.8%	authorship only

239 A.5 Robustness of the same-generator rows

240 At the strict rows’ $n = 99$, B under the identical pipeline is 0.7807 ± 0.0248 . Intervals in the
241 same-generator paragraphs of §3 are bootstraps over problems, resampling S and B' jointly for ρ ;
242 this replaced a ± 2 fold-sd rule before the retry pool was read. Every ρ point we quote is the ratio of
243 the fold-mean AUROCs and every joint interval beside it is the bootstrap of the pooled out-of-fold ρ ,
244 which is the same quantity computed two ways: the pooled points are 0.86 (strict, the row whose
245 interval Table 2 quotes), 0.390 (prompt-matched headline), 0.662 (prompt-matched strict) and 0.097
246 (the second generator). Five fold seeds move the strict row’s ρ over 0.85–0.92. Retention does not
247 produce the effect. Survivors and non-survivors separate at only 0.62, and S computed with non-firing
248 attempts on the same problems is 0.7595 against 0.7687. The edit arm bounds S under near-copies
249 (similarity 0.928, Table A3): 0.6266 over 427 pairs, $\rho = 0.371$. In the one-sided arm of §3, where
250 only the honest half was regenerated, the classifier reaches 0.962 against $B' = 0.874$ on that arm’s
251 $n = 668$ eligible problems, and TF-IDF gains +0.088 while the length carrier collapses to 0.489. Our
252 own attack prompt halves our single-shot firing rate: 52.0% of our test-passing draws offer a public
253 test input as the trigger, which cannot diverge from the honest solution, and those fire 0.15% against
254 5.13% for novel triggers; no stored working backdoor has such a trigger (0 of 3,188). The rest is
255 retry-and-keep.

256 A.6 The prompt-matched arm under the masks, and the second generator

257 Under prompt matching the identifier mask, the only mask of §3 that moved S , takes the headline
258 row to 0.6264 ($\rho = 0.342$) where it takes the original honest class on the same 586 problems to
259 0.7046, so identifier normalisation and prompt matching remove largely the same thing. That masked
260 row is the one that decides whether the residue is structure or naming, and under prompt matching
261 it is band-ambiguous: its joint interval, 0.280–0.394, puts 40% of its mass in the collapse band. A
262 character model (3–5-grams, same classifier and folds) retains more than the word tokeniser, 0.6817
263 ($\rho = 0.493$), so the drop is not an artefact of tokenisation. On the strict pairs the matched row is
264 $S' = 0.7113$ ($n = 96$, $\rho = 0.697$, 0.494–0.856). The second generator’s pairs are more self-similar
265 than the first’s (0.724 mean against 0.659), so S was recomputed inside similarity strata, where it
266 stays 0.4737–0.5142 in every stratum, including the 0.451-similarity stratum, which is further from
267 its honest class than the stored benchmark’s own pairs, while B' holds 0.7884–0.8334, so near-copies
268 are not what buys the collapse. Its prompt-matched rows move S to 0.5399 ($\rho = 0.113$, $n = 562$) and
269 put the prompt-only floor at 0.4918, an interval containing chance. The mechanism is visible in the
270 programs. The layout vocabulary that the first generator’s residue is made of, `def main, sys.stdin`
271 and `data[`, is present in 0.000 of the second generator’s programs in *all three* of its classes, which
272 read standard input with `input(` in 1.000 of programs on every side, so there is no layout difference
273 left for the classifier to read.

274 A.7 The monitor rows, and what bounds the second monitor

275 The monitor that wrote both classes scores 0.566 (0.5507–0.5833) on the same-generator headline
276 pairs, a monitor- ρ of 0.145 (0.111–0.182); on the strict pairs it scores 0.818 (0.7630–0.8686), a
277 monitor- ρ of 0.717 (0.601–0.832). The monitor that wrote neither class scores 0.5577 on the same-
278 generator headline pairs and 0.8824 on the stored pairs over the same problems, 0.7871 and 0.8832
279 on the strict pairs. Its headline rows bound the value without fixing it. A 512-token reply ceiling left
280 628 of 1,246 same-generator replies unparseable, and on the stored side the missing share differs by
281 class (50.1% honest against 23.4% attack), a bias of unknown sign. Re-running the strict pairs with a

1,024-token ceiling prices it: missingness falls to 2 of 198 and monitor- ρ moves from 0.749 to 0.766, inside the bootstrap interval. On the stored pairs both monitors beat the bag-of-words baseline over the same problems (+0.09 and +0.15 for the first, +0.02 and +0.09 for the second): Figure A1’s ranking states something about those monitors on that split and does not claim that a bag of words beats a monitor here. Classifier and monitor are independent pair by pair on the same-generator pairs ($\phi = 0.04$), so the two are reading different things.

A.8 The two generators’ funnel

Both generators were run through the identical pipeline. Our honest draws pass the public tests at 0.7701 and 0.7278 of 1,444 problems; the second generator’s attack draws pass at 689 of 1,444 and 35 of those fire, below the 50-pair minimum `gate_s_samegen.py` requires, leaving it with no strict row and the two generators are compared on the `passes` eligibility only. The two compared rows differ in inertness. 2 of the 611 attacks in the second generator’s row fire (99.67% inert) against 14 of 623 (97.75%), the other 33 of its firing draws sitting on problems where its own honest program failed the tests and which the control therefore drops. The single-shot firing rate belongs to the protocol and not to either model, since 52.0% of test-passing draws offer a public test input as their trigger.

A.9 The five same-generator agentic settings

Removing the authorship gap does not remove every floor. In five settings where the same three models write both classes, so no authorship confound is possible, the surviving floor is structural: 0.504 (bash, on its null) to 0.802 (agentdojo) from character count alone (Figure A2). Figure A2 shows each setting’s content-blind floor against its own permutation null.

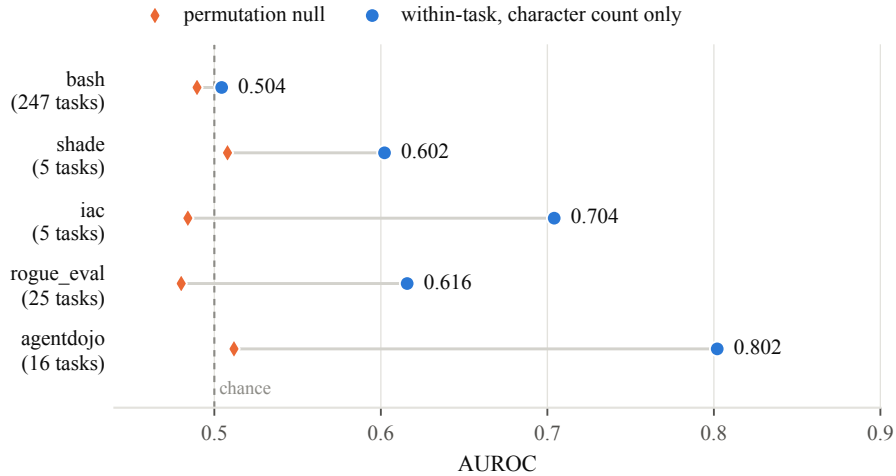


Figure A2: **The structural floor in five same-generator agentic settings.** Each setting’s content-blind classifier (six count features, no content read) scored *within task* against its own permutation null. No authorship confound is possible here: claude-sonnet-4, gemini-2.5-pro and gpt-5 write both classes in all five. bash sits on its null at 0.504; agentdojo reaches 0.802 from character count alone. **The task-group count beside each setting matters:** shade and iac average over five task groups each while bash averages over 247, so these floors are not equally well determined. We report the within-task column because our own pooled numbers partly broke on task composition, and we broke them ourselves. The permutation nulls shown were computed with a pooled estimator rather than the folded within-task statistic; they bound the chance level but are not a like-for-like null for each dot. These corpora declare no licence; we redistribute nothing and report aggregates only.

A.10 The release

The supplementary bundle holds the same-generator split (1,444 honest programs, 1,112 edit-arm and 1,112 independent-arm attack programs written by one model, plus every one of the 11,565

draws of the independent arm’s retry pool), both eligibility rules as pair lists under an outcome-blind selection (623 passing and 14 firing independent-arm pairs) and under the benchmark’s own survivor selection (1,046 and 99, or 106 counting a crash on the trigger as firing), the 24 scripts behind the measurements (generation for both generators, the test harness, the trigger verifier, the selection rules, the prompt-matched arm, the monitor rows and each remaining measurement script), a dataset card, and a manifest of row counts and checksums. The licence, MIT, is a file inside the bundle and covers the data and the scripts alike.

A.11 The test harness and the pass flag

The same-generator arm requires executing generated code, so we built a runner that feeds each shipped input case to a solution under per-case timeouts and address-space limits. It agrees with the artifact’s own `solution_passes_tests` on 95.84% of the analysis pool, and is strictly conservative there, never crediting a solution the artifact rejects (0 such cases). We use *our own* flag for both classes, because a checker stricter on one class than the other would manufacture exactly the artifact this paper documents. We do not length-match either class; the untrusted model chooses its own trajectory length. The headline does not depend on that choice. Under the artifact’s flag $B = 0.871$ at $n = 1,444$, under ours $B = 0.873$ at $n = 1,406$, within one standard deviation of each other. The monitor scores in Figure A1 are `runs/x0/monitor-summary.json` from the MIT release of that harness [7], used under its notice. Two interpreter-era artifacts in the upstream solutions fall on opposite classes: a `stdin` without `.buffer`, fixed, and `fractions.gcd`, removed in Python 3.9, not fixable. B is exposed to them; the same-generator arm, whose code we generated and ran ourselves, is immune.

A.12 The token test, run and failed

We attempted to confirm a reported token-level mechanism [11] in our setting and it does not replicate. Separation is -0.013 , the wrong sign, at permutation $p = 0.956$. The reason is register. Their human class is general-purpose repository code, whereas both of our classes are competitive-programming code, so an identifier axis that separates general human code from model code does not separate these two populations. Only 8 of their 10 canonical tokens are in our vocabulary, so the test is underpowered; but an underpowered true effect would still carry the right sign, and this one does not. We report it because a reader who knows that work may run exactly this test.

A.13 A third channel, noted but not developed here

In the same five agentic settings a fifteen-word regular expression over the untrusted policy’s own scratchpad reaches 0.76–0.82, because prompted attack policies restate their side task in the reasoning the monitor is given. We note this only to bound the claim. The instrument is not ours, and the work it comes from [10] validates it against an LLM judge and a length-matched null; we defer to that treatment.